

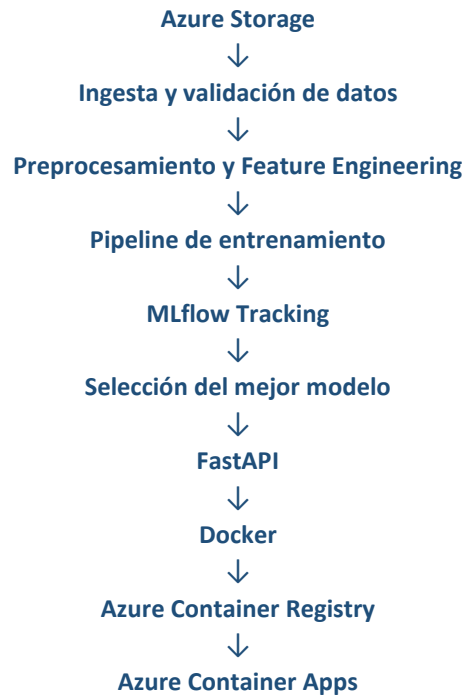
Reporte Técnico

*Prueba Técnica – Ingeniero MLOps / AI Ops
Salva Health*

1. Arquitectura elegida

El objetivo del proyecto fue desarrollar un pipeline MLOps reproducible para entrenar, evaluar y desplegar un modelo de clasificación binaria de registros ECG utilizando variables clínicas y características extraídas de la señal.

La arquitectura implementada separa claramente las etapas del ciclo de vida del modelo:



La solución utiliza Azure Storage para el almacenamiento del dataset, Python para el desarrollo del pipeline, MLflow para el seguimiento de experimentos, FastAPI para exponer el modelo como servicio REST, Docker para garantizar portabilidad y Azure Container Registry junto con Azure Container Apps para el despliegue en la nube.

Esta separación permite desacoplar completamente el entrenamiento del proceso de inferencia, facilitando la actualización del modelo sin modificar la API.

2. Decisiones de versionamiento

La trazabilidad fue uno de los principales criterios de diseño del proyecto.

El código fuente se administró mediante Git, utilizando ramas de desarrollo antes de integrar los cambios a la rama principal. La estructura del proyecto separa los componentes de ingesta, procesamiento, entrenamiento, API y pruebas unitarias, facilitando el mantenimiento y la escalabilidad.

Los datos fueron organizados en las carpetas data/raw y data/processed, conservando los datos originales y almacenando la versión procesada en formato Parquet para reducir tiempos de lectura durante el entrenamiento.

Los modelos entrenados se almacenan como artefactos (artifacts/models/best_model.joblib), mientras que MLflow registra parámetros, métricas, artefactos y resultados de cada experimento, permitiendo identificar de manera reproducible el modelo promovido a producción.

Criterio de selección del modelo

Se entrenaron tres variantes del modelo utilizando diferentes configuraciones de hiperparámetros. Cada experimento fue registrado en MLflow junto con sus parámetros, métricas y artefactos correspondientes.

La promoción del modelo se realizó utilizando la métrica ROC-AUC, ya que evalúa la capacidad del clasificador para discriminar entre pacientes normales y anormales considerando todos los posibles umbrales de decisión. Esto permite comparar diferentes modelos de forma objetiva e independiente del umbral de clasificación.

Aunque métricas como Recall son fundamentales en un contexto clínico para minimizar falsos negativos, se consideran más apropiadas durante la etapa de calibración del umbral de decisión una vez seleccionado el mejor modelo. Bajo este criterio, el modelo con mejor ROC-AUC fue promovido como versión autorizada para producción.

3. Reproducción del pipeline

El proyecto fue diseñado para que cualquier integrante del equipo pueda reproducir el entrenamiento completo mediante un único comando.

El pipeline ejecuta automáticamente:

- Lectura de datos desde Azure Storage.
- Validación de calidad.
- Preprocesamiento.
- Extracción de características ECG.
- Entrenamiento de múltiples variantes.
- Evaluación de modelos.
- Registro de experimentos en MLflow.
- Selección y almacenamiento del mejor modelo.

Dependiendo del objetivo, el proyecto puede ejecutarse mediante:

```
python -m src.pipeline.train_pipeline
```

para reproducir únicamente el entrenamiento, o

```
python -m src.pipeline.run_pipeline
```

para ejecutar el pipeline completo desde la carga de datos hasta la generación del modelo final.

Se decidió utilizar un único punto de entrada mediante `python -m src.pipeline.run_pipeline` en lugar de un Makefile, ya que este enfoque es independiente del sistema operativo, elimina la dependencia de herramientas adicionales y facilita la reproducción del proyecto en cualquier entorno con Python instalado.

Las dependencias están definidas en `requirements.txt`, mientras que las rutas del proyecto son administradas mediante un módulo central de configuración, evitando rutas absolutas y facilitando la ejecución en distintos ambientes.

Como extensión opcional de la prueba también se implementó un pipeline de Integración Continua (CI) mediante GitHub Actions que ejecuta pruebas automáticas y valida el pipeline de entrenamiento en cada cambio del repositorio. Adicionalmente, se desarrolló un proceso de Despliegue Continuo (CD) que construye la imagen Docker, la publica en Azure Container Registry y actualiza automáticamente la aplicación desplegada en Azure Container Apps.

4. Diseño de inferencia (5.7)

El modelo fue expuesto mediante FastAPI debido a su alto rendimiento, validación automática de datos mediante Pydantic y generación de documentación OpenAPI.

El servicio expone un endpoint POST `/predict`, el cual recibe las variables clínicas y las características ECG generadas durante el entrenamiento, realiza la validación del request, carga el modelo autorizado y retorna la clase predicha junto con su probabilidad asociada.

La API incorpora validaciones sobre campos obligatorios, tipos de datos y estructura del request, devolviendo códigos HTTP apropiados cuando se presentan errores de entrada.

El diseño desacopla completamente la API del proceso de entrenamiento. Cuando un nuevo modelo supera al modelo actual, únicamente es necesario actualizar el artefacto autorizado (`best_model.joblib`), manteniendo la misma interfaz de inferencia sin realizar cambios en el código del servicio.

5. Propuesta de monitoreo (5.8)

Aunque el monitoreo no fue implementado como parte del proyecto, se propone una estrategia basada en tres niveles.

En primer lugar, monitorear la disponibilidad del servicio, registrando métricas como latencia, tiempo de respuesta, porcentaje de errores HTTP y estado del endpoint para detectar problemas operativos.

En segundo lugar, monitorear el desempeño del modelo mediante métricas como ROC-AUC, Precision, Recall y F1-score cuando se disponga de etiquetas reales provenientes de producción. Una disminución sostenida en estas métricas activaría un proceso de revisión y reentrenamiento.

Finalmente, implementar monitoreo de Data Drift, comparando la distribución de las variables clínicas, las características ECG y la distribución de predicciones respecto a los datos utilizados durante el entrenamiento. Herramientas como Evidently AI permitirían automatizar esta detección y generar alertas cuando se superen umbrales previamente definidos.

6. Experimento propuesto (5.9)

Si se dispusiera de dos días adicionales, se propondría evaluar arquitecturas capaces de aprender directamente sobre la señal ECG.

Actualmente el pipeline utiliza características estadísticas y energéticas extraídas manualmente para entrenar modelos tradicionales. Como siguiente paso, se compararía este enfoque con una red neuronal convolucional unidimensional (1D-CNN) entrenada sobre la señal ECG cruda y con un modelo híbrido que combine variables clínicas, características manuales y representaciones aprendidas automáticamente.

La comparación utilizaría ROC-AUC, Precision, Recall y F1-score como criterio de evaluación, con el objetivo de determinar si el aprendizaje profundo permite capturar patrones temporales adicionales presentes en la señal y mejorar el desempeño del modelo actual.

7. Limitaciones y trabajo futuro

El proyecto presenta algunas limitaciones inherentes al alcance de la prueba técnica. El dataset corresponde a un subconjunto reducido, por lo que el modelo requiere validación con un volumen mayor de datos antes de un uso en producción. Asimismo, es necesario considerar posibles cambios en la distribución de los datos clínicos una vez el sistema entre en operación.

Como trabajo futuro se propone implementar monitoreo automático del modelo, automatizar la promoción entre ambientes, incorporar búsqueda sistemática de hiperparámetros, ampliar las pruebas de integración y evaluar arquitecturas de Deep Learning para el procesamiento directo de señales ECG.

8. Herramientas de Inteligencia Artificial utilizadas

Durante el desarrollo se utilizaron ChatGPT (OpenAI) y GitHub Copilot como herramientas de apoyo para resolver dudas técnicas relacionadas con Azure, Docker, GitHub Actions, arquitectura MLOps y documentación del proyecto. También fueron empleadas para revisar fragmentos de código, proponer alternativas de implementación y mejorar la redacción de la documentación técnica.

Todas las decisiones relacionadas con la arquitectura, organización del proyecto, implementación del pipeline, selección de tecnologías, entrenamiento del modelo y despliegue fueron analizadas y validadas manualmente antes de incorporarse al desarrollo. Las herramientas de IA fueron utilizadas como apoyo técnico y no sustituyeron el criterio de ingeniería aplicado durante la construcción del proyecto.